AI Design and Implementation of a Multi-Model AI Agent for Automated Health Diagnostics

By: Shivam Kumar Singh

Internship Program: Infosys Springboard Internship 6.0
Institution: Banaras Hindu University
Year: 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Infosys Springboard** for providing me with the opportunity to participate in the Internship 6.0 program. The internship has provided valuable exposure to real-world applications of Artificial Intelligence and practical experience in designing and deploying AI-based systems.

I would also like to thank my mentor, Pranaya, for her guidance, insights, and encouragement throughout the project development process. Their feedback helped refine the system architecture, improve implementation quality, and ensure responsible AI deployment.

I am grateful for the learning resources, technical environment, and structured internship framework provided by Infosys Springboard, which enabled me to successfully complete this project.

Finally, I would like to acknowledge all the open-source tools and libraries used in this project, including Python, Gradio, Hugging Face Spaces, and the Gemini API, which made the development and deployment of this system possible.

Shivam Kumar Singh

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Literature Review
5. Proposed System Architecture
6. Methodology
7. Model 1 – Data Extraction & Parameter Interpretation
8. Model 2 – Dynamic Risk Scoring Using Large Language Models
9. Model 3 – Health Report Synthesis and Natural Language Generation
10. Model 4 – Personalized Contextual Question-Answering Agent
11. Deployment Architecture and System Integration
12. Implementation Details
13. Performance Evaluation and Experimental Results
14. Limitations and Ethical Considerations
15. Future Scope and Research Extensions
16. Conclusion

1. Abstract

The rapid advancement of Artificial Intelligence (AI) has significantly transformed healthcare analytics and decision-support systems. One of the most common yet critical diagnostic tools in clinical practice is the blood test report, which contains multiple biochemical and hematological indicators reflecting the physiological state of a patient. Despite its importance, interpretation of blood reports requires medical expertise, making it difficult for non-medical individuals to understand their health status effectively.

This research presents the design and implementation of a **Multi-Model AI Agent for Automated Health Diagnostics**, capable of extracting, analyzing, and interpreting blood test reports using a modular artificial intelligence framework. The proposed system integrates Optical Character Recognition (OCR), structured data parsing, dynamic Large Language Model (LLM)-based risk assessment, and contextual natural language generation to provide meaningful health insights.

The architecture consists of four primary AI components: (1) Data Extraction and Parameter Classification Model, (2) Dynamic Risk Scoring Model using LLM-based reasoning, (3) Health Report Synthesis Model for structured summary generation, and (4) Personalized Question-Answering Agent incorporating demographic and lifestyle context. The system supports multi-format input including PDF, image, CSV, and JSON files, and is deployed as an interactive web application.

Experimental evaluation demonstrates that the proposed system successfully extracts medical parameters, detects abnormal findings, generates probabilistic risk assessments across multiple disease categories, and provides user-friendly explanations with personalized insights. While not intended to replace medical diagnosis, the system functions as a Clinical Decision Support Tool aimed at enhancing health awareness and facilitating informed medical consultation.

This research highlights the potential of modular AI architectures in healthcare analytics and establishes a scalable framework for future extensions in medical report interpretation systems.

2. Introduction

The integration of Artificial Intelligence (AI) into healthcare systems has significantly enhanced the efficiency, accessibility, and accuracy of diagnostic support tools. In recent years, machine learning and large language models have demonstrated strong capabilities in data interpretation, pattern recognition, and contextual reasoning across diverse domains. One of the most promising applications of AI in healthcare lies in the automated analysis of diagnostic reports, particularly laboratory blood reports.

Blood tests are among the most commonly prescribed medical investigations. They provide quantitative measurements of various physiological parameters, including glucose levels, hemoglobin concentration, lipid profiles, liver enzymes, kidney function indicators, electrolytes, and immune markers. These parameters collectively offer critical insights into metabolic function, cardiovascular health, organ performance, and potential disease conditions.

However, despite their clinical importance, blood reports present several challenges:

1. They contain medical terminology that may not be easily understood by non-professionals.
2. Reference ranges differ based on age, gender, and laboratory standards.
3. Individual parameter deviations do not necessarily indicate disease without contextual interpretation.
4. Correlations between multiple abnormal parameters require expert reasoning.

As a result, patients often rely entirely on healthcare professionals for interpretation. In cases where immediate consultation is not possible, individuals may attempt to interpret reports through online searches, which can lead to misinformation or unnecessary anxiety.

To address these challenges, this research proposes a **Multi-Model AI Agent for Automated Health Diagnostics**, designed to function as an intelligent Clinical Decision Support System (CDSS). The system does not aim to replace medical professionals; rather, it provides structured insights, probabilistic risk estimation, and personalized explanations to improve patient awareness and support informed medical discussions.

Unlike traditional rule-based diagnostic tools, the proposed system integrates:

- Structured data extraction techniques
- Dynamic Large Language Model (LLM)-based reasoning
- Modular architecture for scalability
- Context-aware personalized interaction

The core innovation of this research lies in its **multi-model architecture**, where each model performs a distinct but interconnected task:

- Extraction and classification of parameters
- Dynamic disease risk scoring
- Natural language synthesis of medical insights
- Personalized question-answering

This modular design ensures flexibility, interpretability, and extensibility, making the system adaptable to future enhancements such as additional diagnostic modalities (e.g., ECG, urine tests, radiology reports).

The system is deployed as a web-based application using modern AI deployment infrastructure, enabling real-time processing and interactive engagement. Through this research, we demonstrate how AI-driven architectures can bridge the gap between raw medical data and meaningful health understanding for non-expert users.

3. Problem Statement

Healthcare diagnostics rely extensively on laboratory investigations, with blood tests serving as one of the most fundamental diagnostic tools. These reports contain multiple quantitative measurements representing various physiological and biochemical functions. While medical professionals are trained to interpret these parameters in relation to clinical context, patients often struggle to understand the implications of their test results.

The primary problem addressed in this research can be formally stated as follows:

There is a lack of accessible, intelligent, and automated systems capable of interpreting unstructured blood test reports and providing structured, contextualized, and personalized health insights to non-medical users.

Several sub-problems contribute to this overarching issue:

3.1 Complexity of Medical Reports

Blood reports contain numerous parameters, including but not limited to glucose levels, hemoglobin concentration, lipid profile components, liver enzymes, kidney markers, and immune indicators. Each parameter has a reference range, and deviations may indicate potential health risks. However:

- Patients often do not understand the significance of “High” or “Low” values.
- Multiple abnormal parameters may interact in complex ways.
- Individual interpretation requires domain knowledge.

3.2 Variability in Report Formats

Blood reports are not standardized globally. They may be:

- Text-based PDFs,
- Scanned image reports,
- CSV exports from laboratory systems,
- JSON-based structured files.

This variability introduces technical challenges in automated extraction and structured representation.

3.3 Limitations of Rule-Based Systems

Existing automated tools often rely on rule-based systems, which:

- Require manual updates for new parameters,
- Cannot dynamically reason across correlated parameters,

- Lack contextual understanding,
- Fail to adapt to complex medical patterns.

A static rule-based approach cannot scale effectively in real-world healthcare scenarios.

3.4 Absence of Personalized Interpretation

Health interpretation is influenced by demographic and contextual factors such as:

- Age,
- Gender,
- Lifestyle,
- Symptoms,
- Medical history.

Most existing tools do not incorporate these contextual variables into their reasoning process.

3.5 Lack of Interactive Support

Even when reports are summarized, users may still have questions. Traditional systems do not provide interactive follow-up explanations tailored to individual circumstances.

3.6 Research Gap

From the analysis above, the research gap can be identified as:

- The absence of a modular, AI-driven, scalable system capable of:
    - Extracting structured medical parameters from unstructured reports,
    - Dynamically estimating disease risk using large language models,
    - Synthesizing comprehensive health summaries,
    - Providing contextual, personalized question-answering capabilities.

This research aims to fill this gap through the design and implementation of a Multi-Model AI Agent architecture.

3.7 Scope of the Proposed Solution

The proposed system addresses the problem by:

1. Automating parameter extraction across multiple input formats.

2. Classifying abnormal values systematically.
3. Leveraging LLM-based reasoning for dynamic risk scoring.
4. Generating user-friendly health summaries.
5. Enabling personalized interaction through a contextual chatbot.
6. Deploying the system online for real-time accessibility.

It is important to emphasize that the system functions as a **decision-support tool**, not a diagnostic replacement. It assists users in understanding their reports and encourages professional consultation when necessary.

4. Literature Review

The application of Artificial Intelligence (AI) in healthcare has expanded significantly over the past decade, particularly in areas such as medical imaging, predictive diagnostics, clinical decision support systems (CDSS), and automated report analysis. This section reviews relevant concepts and existing approaches that form the foundation of the proposed system.

4.1 Artificial Intelligence in Healthcare

AI technologies, particularly machine learning (ML) and deep learning (DL), have been widely adopted for medical diagnostics, including:

- Disease prediction using structured health data
- Radiology image analysis
- Clinical outcome forecasting
- Personalized treatment recommendation

Research indicates that AI systems can assist clinicians by improving diagnostic efficiency and reducing human error. However, most AI systems are trained on structured datasets and do not directly address the challenge of interpreting unstructured medical reports for non-expert users.

4.2 Clinical Decision Support Systems (CDSS)

Clinical Decision Support Systems are computer-based systems designed to assist healthcare providers in making clinical decisions. Traditional CDSS implementations:

- Use rule-based logic
- Depend on predefined medical knowledge bases
- Operate within hospital infrastructures

Limitations of traditional CDSS include:

- Lack of scalability
- Difficulty updating medical rules
- Limited contextual reasoning
- Restricted accessibility for public users

The proposed system extends CDSS principles by integrating dynamic AI reasoning and making it accessible through a web-based interface.

4.3 Automated Medical Report Interpretation

Several studies have explored automated interpretation of medical data. Common approaches include:

1. Rule-Based Systems
    - Compare parameter values to fixed thresholds
    - Generate pre-written alerts
    - Limited contextual intelligence
2. Machine Learning Models
    - Predict disease probability from structured datasets
    - Require large labeled datasets
    - Often lack interpretability
3. Hybrid Systems
    - Combine structured data extraction with probabilistic models
    - Improve flexibility but still rely on predefined disease mappings

A major limitation of these systems is their inability to dynamically reason across multiple parameters without explicit programming.

4.4 Optical Character Recognition (OCR) in Medical Applications

OCR has been widely used for digitizing medical documents. Tesseract OCR is one of the most commonly used open-source OCR engines.

Challenges in medical OCR include:

- Variability in report formatting
- Low image quality
- Medical terminology recognition

In this research, OCR is used as a preprocessing tool to convert scanned reports into machine-readable text before structured parsing.

4.5 Large Language Models (LLMs) in Healthcare

Large Language Models such as GPT-based and Gemini-based systems have demonstrated advanced capabilities in:

- Contextual reasoning
- Medical text summarization
- Question answering
- Pattern recognition in structured data

Recent research highlights the ability of LLMs to:

- Perform zero-shot reasoning
- Generate structured outputs (JSON)
- Interpret medical parameters contextually

Unlike rule-based systems, LLMs can dynamically infer relationships between multiple variables without explicit hardcoding.

However, challenges include:

- Ensuring output reliability
- Avoiding hallucinations
- Maintaining ethical boundaries
- Including disclaimers

The proposed system leverages LLM capabilities while maintaining strict instruction control and structured output formatting.

4.6 Research Gap Analysis

From the literature review, the following gap is identified:

- Most systems focus either on structured dataset-based prediction or static rule-based alerts.
- Few systems combine:
    - Multi-format ingestion,
    - Structured extraction,
    - LLM-based dynamic risk scoring,
    - Natural language synthesis,

- Personalized contextual chatbot interaction,
- Online deployment in a unified architecture.

The proposed research bridges this gap through a modular multi-model design integrating both deterministic logic and generative AI reasoning.

4.7 Summary of Literature Findings

The review establishes that:

1. AI has strong potential in medical data interpretation.
2. LLMs provide scalable reasoning beyond rule-based systems.
3. OCR enables digitization of unstructured medical reports.
4. A modular AI architecture improves maintainability and extensibility.
5. There is limited work integrating all these components into a single accessible system.

The present research contributes by implementing an integrated, scalable, and deployable AI health diagnostics agent.

5. Proposed System Architecture

The proposed system is designed as a **Multi-Model Modular AI Architecture** for automated blood report interpretation and health risk assessment. The architecture follows a layered design pattern to ensure modularity, scalability, maintainability, and clarity of functionality.

The system is divided into three major layers:

1. Input Layer
2. Processing Layer (Multi-Model AI Core)
3. Output & Interaction Layer

Each layer performs a distinct function within the end-to-end workflow.

5.1 High-Level Architectural Overview

The workflow of the proposed system can be summarized as:

User Upload → Text Extraction → Parameter Structuring → Risk Analysis → Report Synthesis → Personalized Interaction → PDF Generation

The modular separation ensures that each component can operate independently while contributing to the overall system pipeline.

5.2 Input Layer

The Input Layer is responsible for accepting user-uploaded medical reports in various formats.

Supported File Formats:

- PDF (text-based reports)
- Image (scanned reports: PNG, JPG, JPEG)
- CSV (laboratory exports)
- JSON (structured lab data)

Functional Responsibilities:

1. File type detection
2. Routing to appropriate extraction module
3. Preprocessing for downstream parsing

This layer ensures flexibility and real-world usability, as laboratory reports are generated in diverse formats.

5.3 Processing Layer (Multi-Model AI Core)

The Processing Layer constitutes the core intelligence of the system. It is composed of four interlinked models.

5.3.1 Model 1: Data Extraction & Parameter Classification

Purpose:

Convert unstructured report content into structured medical data.

Sub-components:

- PDF Text Extractor (pdfplumber)
- OCR Engine (pytesseract)
- Regex-based Parser
- Range Comparator

Workflow:

1. Extract raw text.
2. Identify patterns:
    - Parameter Name
    - Tested Value
    - Reference Range
3. Parse into structured format.
4. Compare value against reference limits.
5. Assign status:
    - Normal
    - High
    - Low

Output Format (Structured JSON):

```
{
  "Glucose": {
    "value": 141,
    "reference": "74 - 106",
    "status": "High"
  }
}
```


This structured representation forms the basis for AI reasoning.

5.3.2 Model 2: Dynamic Risk Scoring Engine

Purpose:

Estimate disease risk probabilities dynamically using Large Language Model reasoning.

Design Principle:

Unlike traditional rule-based systems, this model does not use hardcoded medical logic. Instead, it sends structured JSON data to a Large Language Model (Gemini API) with carefully engineered prompts.

Processing Steps:

1. Provide parameter statuses to LLM.
2. Request structured JSON output including:
    - Risk score (0–100)
    - Risk level (Low / Moderate / High)
    - Justification
3. Parse and validate LLM response.

Risk Categories Evaluated:

- Diabetes Risk
- Cardiovascular Risk
- Kidney Risk
- Liver Risk
- Anemia Risk

Advantages:

- Scalable to new diseases
- No manual rule updates required
- Context-aware pattern recognition
- Multi-parameter correlation reasoning

5.3.3 Model 3: Health Report Synthesis Module

Purpose:

Convert technical outputs into a coherent, human-readable medical summary.

Inputs:

- Model 1 structured parameter analysis
- Model 2 risk scoring JSON

Outputs:

- Overall health summary
- Top critical findings
- Risk dashboard explanation
- Personalized recommendations
- Follow-up test suggestions
- Doctor consultation advice
- Disclaimer

This model ensures interpretability and accessibility of AI outputs.

5.3.4 Model 4: Personalized Q&A Agent

Purpose:

Provide contextual, demographic-aware responses to user questions.

Additional Inputs:

- Age
- Gender
- Lifestyle habits

- Symptoms
- User query

Functional Role:

The agent uses:

- Structured report data
- Risk scoring data
- Final synthesized report
- User profile

to generate personalized responses.

This transforms the system from a static report generator into an interactive AI health assistant.

5.4 Output & Interaction Layer

The Output Layer ensures that results are presented in an accessible and user-friendly manner.

Components:

1. Parameter Table Display
2. Risk Dashboard (0–100 sliders)
3. Final Health Report (structured text)
4. Downloadable PDF generation
5. Interactive Chat Interface

Design Goals:

- Clarity
- Visual interpretability
- Accessibility
- Responsiveness

5.5 Architectural Design Principles

The system architecture is built on the following principles:

1. Modularity

Each model operates independently and can be upgraded without affecting others.

2. Scalability

New diagnostic categories can be added by extending Model 2 prompts.

3. Extensibility

Future integration of ECG, radiology reports, or wearable data is possible.

4. Interpretability

Outputs are structured and human-readable.

5. Deployment Compatibility

Architecture supports cloud-based hosting (Hugging Face Spaces).

5.6 Architectural Advantages Over Traditional Systems

Traditional Rule-Based System	Proposed Multi-Model AI System
Static rules	Dynamic LLM reasoning
Hardcoded disease mapping	Scalable risk estimation
Limited contextual awareness	Personalized demographic reasoning
No interaction	Interactive chatbot

This architecture establishes a robust foundation for automated health diagnostics using AI-driven modular design.

6. Methodology

The methodology of this research is centered around the systematic transformation of unstructured medical data into structured, interpretable, and personalized health insights through a modular AI-driven pipeline. The methodology is divided into six major phases:

1. Data Ingestion
2. Text Extraction
3. Parameter Parsing & Structuring
4. Abnormality Detection
5. Dynamic Risk Assessment
6. Report Synthesis & Personalized Interaction

Each phase is explained in detail below.

6.1 Phase 1: Data Ingestion

The first step involves accepting user-uploaded blood reports in multiple formats. The system supports:

- PDF documents
- Image files (PNG, JPG, JPEG)
- CSV files
- JSON files

Algorithmic Flow:

1. User uploads file via web interface.
2. System detects file type using extension.
3. File routed to appropriate extraction module.
4. Raw file content prepared for processing.

This ensures flexibility and real-world compatibility.

6.2 Phase 2: Text Extraction

The objective of this phase is to convert raw file content into machine-readable text.

Case 1: PDF Reports

- Use pdfplumber to extract textual content directly.
- Clean unnecessary whitespace and formatting characters.

Case 2: Image Reports

- Use pytesseract (Tesseract OCR engine).
- Convert image to grayscale (optional preprocessing).
- Extract text from scanned document.
- Perform text normalization.

Case 3: CSV / JSON

- Direct structured parsing using Pandas.
- Convert into uniform internal format.

The output of this phase is raw text containing parameter data.

6.3 Phase 3: Parameter Parsing & Structuring

This phase converts raw text into structured medical data.

Step 1: Pattern Detection

Using Regular Expressions (Regex), the system identifies patterns matching:

- Parameter name
- Tested value
- Reference range

Example pattern format:

ParameterName TestedValue ReferenceRange

Step 2: Data Structuring

Each parameter is converted into a structured JSON object:

```
{
  "Hemoglobin": {
    "value": 10.5,
    "reference_low": 12.0,
    "reference_high": 15.5
```

```
  }
}
```

Step 3: Data Cleaning

- Remove duplicate entries.
- Convert numeric strings to float values.
- Handle missing reference ranges.

This structured representation enables downstream AI reasoning.

6.4 Phase 4: Abnormality Detection (Model 1 Logic)

For each parameter:

If value < lower limit → Status = "Low"
If value > upper limit → Status = "High"
Else → Status = "Normal"

Mathematical Representation:

Let:

V = Tested value
L = Lower reference limit
H = Upper reference limit

Then:

Status(V) =

- Low, if $V < L$
- High, if $V > H$
- Normal, if $L \leq V \leq H$

This deterministic classification forms the base health interpretation.

6.5 Phase 5: Dynamic Risk Assessment (Model 2)

Unlike deterministic classification, disease risk estimation requires multi-parameter reasoning.

Approach Used:

Large Language Model (Gemini API) with structured prompting.

Steps:

1. Provide structured JSON from Model 1.
2. Prompt LLM to:
    - Analyze parameter correlations.
    - Compute risk scores (0–100).
    - Assign qualitative risk levels.
    - Provide reasoning.
3. Receive structured JSON response.
4. Validate response format.
5. Extract risk categories.

Risk Score Interpretation:

0–20 → Very Low
21–40 → Low
41–60 → Moderate
61–80 → High
81–100 → Very High

This dynamic reasoning eliminates rigid rule-based systems.

6.6 Phase 6: Report Synthesis (Model 3)

The goal of this phase is interpretability.

Inputs:

- Structured parameter classification
- Risk scoring output

Processing:

- Identify top abnormal findings.
- Prioritize highest risk category.
- Generate actionable recommendations.
- Suggest doctor specialization.
- Add ethical disclaimer.

The output is a readable medical summary.

6.7 Phase 7: Personalized Question Answering (Model 4)

This phase introduces contextual personalization.

Additional Inputs:

- Age
- Gender
- Lifestyle habits
- Symptoms
- User query

Processing Logic:

1. Combine:
    - Parameter data
    - Risk scores
    - Final report
    - User profile
2. Send to LLM with contextual prompt.
3. Generate personalized response.
4. Ensure non-diagnostic language.

This phase converts the system into an interactive AI assistant.

6.8 PDF Generation & Output Delivery

After report generation:

- Use ReportLab to generate downloadable PDF.
- Display risk scores using slider dashboard.
- Show structured table of parameters.
- Enable chat interaction.

6.9 Overall Algorithm Summary

The complete system algorithm can be summarized as:

1. Upload file
2. Extract text
3. Parse parameters
4. Classify values
5. Generate risk scores
6. Synthesize final report
7. Provide personalized Q&A
8. Generate downloadable PDF

6.10 Methodological Strengths

- Modular design ensures maintainability.
- Hybrid approach combines deterministic and generative AI.
- Supports real-world variability in report formats.
- Scalable to additional disease categories.
- Fully deployable cloud-based solution.

7. Model 1: Data Extraction and Parameter Interpretation

Model 1 forms the foundational layer of the proposed Multi-Model AI Agent. Its primary objective is to transform unstructured or semi-structured medical report data into a structured representation suitable for computational reasoning.

This model performs two major tasks:

1. Text Extraction from uploaded files
2. Parameter Classification based on reference ranges

7.1 Objectives of Model 1

The objectives of Model 1 are:

- To extract textual information from various input formats.
- To detect medical parameter entries automatically.
- To structure extracted information into machine-readable format.
- To classify parameter values as Normal, High, or Low.
- To prepare standardized output for downstream AI modules.

7.2 Input Variability Handling

Blood reports can appear in different formats. Model 1 includes format-aware preprocessing:

Case A: PDF Documents

- Text-based PDFs are parsed using pdfplumber.
- Extracted text is cleaned for consistent formatting.

Case B: Image-Based Reports

- OCR is performed using pytesseract.
- Images are converted to readable text.
- Noise reduction and whitespace normalization applied.

Case C: CSV Files

- Parsed using pandas.

- Direct mapping of columns to parameter-value pairs.

Case D: JSON Files

- Direct structured ingestion.
- Converted into internal uniform format.

This multi-format support enhances robustness and real-world applicability.

7.3 Text Preprocessing

After extraction, raw text may contain:

- Extra whitespace
- Line breaks
- Special characters
- Formatting artifacts

Preprocessing steps include:

- Whitespace normalization
- Removal of non-printable characters
- Standardization of decimal formatting
- Cleaning of irrelevant text sections

This ensures consistent pattern detection.

7.4 Parameter Parsing Using Regular Expressions

The core logic of Model 1 relies on pattern recognition using Regular Expressions (Regex).

A typical blood report entry follows the structure:

Parameter Name Tested Value Reference Range

Example:

Glucose 141 mg/dL 74 – 106

Regex Matching Strategy:

The parser searches for patterns that include:

- Alphabetic parameter names

- Numeric tested values
- Numeric range boundaries

Extracted Components:

For each detected entry:

- Parameter Name (String)
- Tested Value (Float)
- Reference Lower Limit (Float)
- Reference Upper Limit (Float)

7.5 Structured Data Representation

After extraction, each parameter is converted into a structured JSON-like format:

```
{
  "Glucose": {
    "value": 141.0,
    "reference_low": 74.0,
    "reference_high": 106.0,
    "status": "High"
  }
}
```

This structured representation is critical for:

- Standardized AI input
- Risk model compatibility
- Report synthesis consistency

7.6 Abnormality Classification Algorithm

The abnormality detection logic follows a deterministic threshold comparison.

Let:

V = Tested value
L = Lower reference limit
H = Upper reference limit

Then classification is defined as:

If V < L → Status = Low
If V > H → Status = High
Else → Status = Normal

Algorithm Pseudocode:

```
For each parameter:
   If value < lower_limit:
      status = "Low"
   Else if value > upper_limit:
      status = "High"
   Else:
      status = "Normal"
```

This classification provides binary health deviation detection without diagnosing disease.

7.7 Handling Missing or Irregular Data

Real-world reports may contain:

- Missing reference ranges
- Units in varying formats
- Multi-line parameter entries

To handle this:

- Units are ignored during numeric extraction.
- Missing ranges are flagged.
- Only parameters with valid ranges are classified.
- Error handling ensures pipeline continuity.

7.8 Model Output Characteristics

Model 1 output is:

- Structured
- Deterministic
- Transparent
- Explainable

- Non-generative

Unlike downstream models, Model 1 does not rely on probabilistic AI reasoning. It ensures baseline correctness before higher-level inference.

7.9 Strengths of Model 1

- Robust multi-format handling
- Transparent classification logic
- Easily extensible parsing patterns
- Deterministic reliability
- Structured output compatibility

7.10 Limitations of Model 1

- OCR accuracy depends on image quality.
- Unusual parameter naming may reduce parsing accuracy.
- Complex multi-table reports may require enhanced regex logic.

Despite these limitations, Model 1 provides a strong structured foundation for dynamic risk estimation.

8. Model 2: Dynamic Risk Scoring Using Large Language Models

Model 2 represents the core intelligence layer of the proposed system. While Model 1 performs deterministic classification of parameters, Model 2 introduces probabilistic reasoning and multi-parameter correlation analysis using a Large Language Model (LLM).

The objective of this model is to estimate disease risk dynamically without relying on hardcoded medical rules.

8.1 Motivation for Using LLM-Based Risk Scoring

Traditional diagnostic support systems often use rule-based mappings such as:

- If Glucose > X → Diabetes risk
- If LDL > Y → Cardiovascular risk

However, such systems suffer from:

- Limited scalability
- Inability to reason across multiple correlated parameters
- High maintenance cost for rule updates
- Lack of contextual explanation

Large Language Models (LLMs) provide advanced capabilities including:

- Multi-variable reasoning
- Contextual understanding
- Natural language explanation
- Structured JSON output generation

Therefore, Model 2 replaces static rule-based inference with dynamic LLM reasoning.

8.2 Input to Model 2

Model 2 receives structured output from Model 1 in JSON format:

Example:

```
{
  "Glucose": {"value": 141, "status": "High"},
  "Hemoglobin": {"value": 10.5, "status": "Low"},
```

"LDL": {"value": 165, "status": "High"}
}

This structured representation enables the LLM to reason across multiple abnormal findings simultaneously.

8.3 Risk Categories Defined

The system evaluates risk across five major health categories:

1. Diabetes Risk
2. Cardiovascular Risk
3. Kidney Risk
4. Liver Risk
5. Anemia Risk

These categories were selected because:

- They are commonly reflected in blood reports.
- They represent major global health concerns.
- They are clinically relevant and interpretable.

8.4 Prompt Engineering Strategy

The performance of LLM-based systems heavily depends on prompt design. Therefore, a structured prompt engineering strategy was implemented.

The prompt includes:

- Structured Model 1 output
- Clear instructions
- Risk scoring scale definition (0–100)
- Output format requirement (strict JSON)
- Constraints to avoid diagnosis claims
- Instruction to include reasoning

Key Prompt Instructions:

- Compute risk score between 0–100.

- Assign qualitative level (Low / Moderate / High / Very High).
- Provide brief medical reasoning.
- Do not provide final diagnosis.
- Return valid JSON only.

This structured instruction ensures:

- Controlled LLM behavior
- Reduced hallucination risk
- Reliable structured output

8.5 Risk Score Interpretation Scale

The numeric risk score is mapped to qualitative categories:

0–20 → Very Low
21–40 → Low
41–60 → Moderate
61–80 → High
81–100 → Very High

This dual representation improves interpretability.

8.6 Multi-Parameter Correlation Reasoning

One of the major advantages of LLM reasoning is its ability to detect correlations.

For example:

- High fasting glucose + high HbA1c → Elevated diabetes risk
- High LDL + high triglycerides → Cardiovascular risk
- Low hemoglobin + low RBC count → Anemia risk

Instead of manually coding these correlations, the LLM infers them dynamically.

This provides flexibility and scalability.

8.7 Output Structure

The model returns structured JSON:

```
{
  "risk_scores": {
    "diabetes": {
      "score": 85,
      "level": "High",
      "reason": "Elevated fasting glucose significantly increases diabetes risk."
    },
    "cardiovascular": {
      "score": 70,
      "level": "High",
      "reason": "Elevated LDL contributes to cardiovascular risk."
    }
  }
}
```


This structured output is:

- Machine-readable
- Compatible with dashboard visualization
- Easily integrated into Model 3

8.8 Advantages of Dynamic LLM-Based Risk Scoring

1. Eliminates rigid rule-based logic
2. Automatically adapts to new parameters
3. Generates contextual reasoning
4. Scalable across disease categories
5. Supports structured output formatting

8.9 Limitations and Mitigation Strategies

Limitation 1: Hallucination Risk

Mitigation:

- Strict JSON output enforcement
- Controlled prompt constraints

Limitation 2: Non-deterministic outputs

Mitigation:

- Clear numeric scale definition
- Output validation before use

Limitation 3: Ethical Risk (Diagnosis claims)

Mitigation:

- Explicit instruction to avoid diagnosis
- Mandatory disclaimer added in Model 3

8.10 Model 2 as a Probabilistic Inference Layer

It is important to emphasize that Model 2 does not diagnose disease. Instead, it:

- Estimates probabilistic risk
- Identifies potential patterns
- Provides explanatory context

Thus, Model 2 functions as an AI-powered probabilistic inference engine within the system.

8.11 Contribution of Model 2 to the Research

Model 2 transforms the system from a simple abnormal value detector into a true AI-driven health analytics system.

It demonstrates:

- Practical use of LLM for structured medical reasoning
- Modular integration of generative AI in CDSS
- Real-world deployment feasibility

9. Model 3: Health Report Synthesis and Natural Language Generation

Model 3 functions as the interpretability layer of the proposed Multi-Model AI Agent. While Model 1 produces structured abnormality classifications and Model 2 generates probabilistic risk scores, these outputs are still technical and may not be easily understandable to non-medical users.

The primary objective of Model 3 is to synthesize technical outputs into a coherent, human-readable health report using natural language generation (NLG) techniques powered by a Large Language Model.

9.1 Motivation for Report Synthesis

Even after risk scores are computed, users may still face challenges:

- Numeric risk values lack contextual explanation.
- Multiple abnormal parameters require prioritization.
- Users need actionable guidance.
- Technical terminology may cause confusion.

Thus, Model 3 bridges the gap between AI computation and user interpretability.

9.2 Inputs to Model 3

Model 3 receives:

1. Structured parameter classification from Model 1
2. Risk scoring JSON from Model 2

These inputs include:

- Parameter names
- Tested values
- Reference ranges
- Abnormal status
- Risk scores (0–100)
- Risk levels
- Reasoning

9.3 Core Functional Responsibilities

Model 3 performs the following tasks:

1. Summarizes overall health status
2. Identifies top critical findings
3. Prioritizes highest risk category
4. Generates actionable lifestyle recommendations
5. Suggests follow-up medical tests
6. Recommends doctor specializations
7. Adds ethical disclaimer

This ensures comprehensive and structured reporting.

9.4 Natural Language Generation Strategy

The system uses structured prompt engineering to guide the LLM in generating a report with predefined sections:

- Overall Health Summary
- Key Abnormal Findings
- Risk Score Overview
- Lifestyle Recommendations
- Follow-Up Test Suggestions
- Doctor Consultation Advice
- Disclaimer

The prompt explicitly instructs:

- Use simple, non-alarming language.
- Avoid diagnostic claims.
- Provide supportive tone.
- Maintain structured output format.

This ensures controlled and responsible AI behavior.

9.5 Report Structure

The synthesized report typically includes:

1. Overall Health Summary

A concise summary describing general condition based on detected abnormalities.

2. Top Critical Findings

Highlights the most significant abnormal parameters.

3. Risk Dashboard Explanation

Describes risk levels in simple language.

4. Personalized Recommendations

Includes:

- Diet suggestions
- Exercise recommendations
- Lifestyle modifications

5. Follow-Up Tests

Suggests relevant diagnostic tests when needed.

6. Doctor Consultation Suggestions

Examples:

- Endocrinologist
- Cardiologist
- Hematologist

7. Disclaimer

Clarifies that the system does not provide medical diagnosis.

9.6 Interpretability and Explainability

One of the major contributions of Model 3 is improving AI explainability.

Instead of providing:

"Diabetes Risk: 85"

The system explains:

"Your fasting blood sugar is significantly elevated, which is commonly associated with increased diabetes risk. It is advisable to consult an endocrinologist."

This improves:

- User understanding
- Trust in AI output
- Responsible health guidance

9.7 Ethical Safeguards in Model 3

To ensure ethical compliance:

- The model avoids definitive diagnostic statements.
- Language is supportive, not alarming.
- A disclaimer is always appended.
- Recommendations encourage professional consultation.

This maintains responsible AI deployment in healthcare.

9.8 Advantages of Model 3

1. Converts technical outputs into understandable language.
2. Enhances user trust and clarity.
3. Provides structured actionable guidance.
4. Maintains ethical and non-diagnostic boundaries.
5. Improves accessibility for non-medical users.

9.9 Limitations

- Generated text may vary slightly due to LLM non-determinism.
- Requires strict prompt control to avoid medical overreach.
- Depends on the quality of Model 1 and Model 2 outputs.

Despite these limitations, Model 3 significantly enhances usability and interpretability.

9.10 Role of Model 3 in the Overall Architecture

Model 3 transforms the system from a technical risk calculator into a comprehensive AI health assistant.

It acts as:

- An AI explanation engine
- A health insight summarizer
- A responsible recommendation generator

Thus, it forms the interpretability backbone of the entire system.

10. Model 4: Personalized Contextual Question-Answering Agent

Model 4 transforms the system from a static report generator into an interactive AI health assistant. While Models 1, 2, and 3 provide structured analysis and summary, Model 4 enables contextual, personalized interaction between the user and the system.

This module leverages Large Language Model (LLM)-based reasoning to answer user queries dynamically, using both report data and demographic context.

10.1 Motivation for Personalized Q&A

Even after receiving a synthesized health report, users may still have questions such as:

- "Is my glucose dangerous for my age?"
- "Can I continue gym with this cholesterol level?"
- "Do I need to worry about anemia?"
- "Which doctor should I consult first?"

Traditional systems stop at report generation. They do not allow follow-up clarification. Model 4 addresses this gap by introducing interactive AI-driven dialogue.

10.2 Inputs to Model 4

Model 4 combines multiple data sources:

1. Structured Parameter Data (Model 1 Output)

Includes parameter names, values, and abnormal status.

2. Risk Scoring Output (Model 2)

Includes:

- Risk scores
- Risk levels
- Justifications

3. Final Synthesized Report (Model 3)

Provides contextual explanation and recommendations.

4. User Profile Information

Includes:

- Age
- Gender
- Lifestyle habits (smoking, alcohol, exercise)
- Symptoms

5. User Question

Natural language query entered by user.

10.3 Context-Aware Reasoning Strategy

The model integrates:

- Structured medical data
- Risk analysis
- Personalized demographics
- User intent

This allows context-sensitive responses.

For example:

Case 1:
Age: 22
Glucose: Slightly High
Response: Focus on lifestyle modification, lower immediate concern.

Case 2:
Age: 65
Glucose: Slightly High
Response: Greater emphasis on cardiovascular risk and monitoring.

Thus, age and gender influence contextual interpretation.

10.4 Prompt Engineering for Controlled Behavior

The prompt provided to the LLM includes:

- Structured medical data
- Risk scoring JSON
- User profile

- The specific user question

The instructions enforce:

- No diagnostic claims
- Clear, supportive language
- Actionable advice
- Encourage doctor consultation if risk is high

This ensures safe and responsible output.

10.5 Functional Responsibilities of Model 4

Model 4 performs:

1. Personalized explanation
2. Clarification of risk severity
3. Lifestyle-based recommendations
4. Symptom-based contextual advice
5. Follow-up guidance
6. Non-alarming reassurance

10.6 Example Interaction Flow

User Input:
"I am 24 years old male. My fasting sugar is 141. Should I be worried?"

Model Processing:

- Detects high glucose from Model 1
- Reads diabetes risk score from Model 2
- Considers age (young adult)
- Generates contextual response

Output Example:
"Your fasting blood sugar level is elevated, which increases your risk for diabetes. Since you are 24, early lifestyle adjustments such as diet control and regular exercise may significantly reduce long-term risk. However, it is advisable to consult a physician for further evaluation."

This response is personalized and actionable.

10.7 Distinction from Generic Chatbots

Unlike general-purpose chatbots:

- This system grounds responses in user-specific medical data.
- It uses structured report context.
- It integrates demographic information.
- It is constrained by health-specific instructions.

Thus, it functions as a contextual health reasoning agent rather than a general conversational AI.

10.8 Ethical Safeguards

To ensure safe healthcare AI usage:

- Avoids medical diagnosis statements.
- Avoids prescribing medication.
- Includes advisory language.
- Encourages professional consultation.
- Prevents fear-inducing statements.

This ensures responsible AI deployment in healthcare context.

10.9 Technical Significance of Model 4

Model 4 represents:

- Contextual AI reasoning
- Multi-source data integration
- Interactive healthcare assistance
- Dynamic conversational intelligence

It significantly enhances system usability and user engagement.

10.10 Contribution to Research Innovation

The inclusion of a personalized contextual Q&A module:

- Demonstrates practical integration of LLMs in healthcare.
- Shows how generative AI can augment decision-support systems.
- Extends static analytics into interactive AI systems.
- Enhances human-AI collaboration in medical interpretation.

11. Deployment Architecture and System Integration

The proposed Multi-Model AI Agent is designed not only as a conceptual framework but as a fully functional, cloud-deployed application. This section describes the deployment strategy, system integration mechanisms, API management, and infrastructure design.

11.1 Deployment Environment

The system is deployed using **Hugging Face Spaces**, which provides:

- Cloud-based hosting
- Gradio integration
- Automatic build environment
- Free-tier deployment capability
- Containerized execution

This deployment approach eliminates the need for local installation and allows real-time public access.

11.2 System Architecture in Deployment Context

The deployed system consists of the following integrated components:

1. Frontend Layer (Gradio Interface)
2. Backend Processing Engine (Python-based pipeline)
3. AI Inference Layer (Gemini API)
4. OCR Engine (Tesseract)
5. PDF Generation Module (ReportLab)
6. Cloud Hosting Environment (HF Spaces)

Each component communicates within a single execution container.

11.3 Frontend Interface (Gradio)

The frontend is implemented using Gradio Blocks, providing:

- Tab-based UI structure
- File upload interface

- Risk dashboard visualization
- Markdown report display
- Interactive chatbot input/output
- PDF download button

Gradio was chosen because:

- It integrates seamlessly with Hugging Face Spaces.
- It supports state management.
- It allows rapid prototyping of AI applications.

11.4 Backend Processing Engine

The backend is implemented entirely in Python and follows a modular file structure:

- file_parser.py → Handles text extraction
- model.py → Performs abnormality detection
- gemini_agent.py → Handles risk scoring
- synthesis_agent.py → Generates final report
- qa_agent.py → Personalized Q&A module
- pdf_generator.py → PDF report generation
- app.py → Main orchestration and UI logic

This modular design ensures:

- Separation of concerns
- Maintainability
- Scalability
- Ease of debugging

11.5 API Integration (Gemini LLM)

The system integrates Google Gemini API for:

- Risk scoring
- Report synthesis

- Personalized Q&A

API Integration Flow:

1. Structured JSON prepared.
2. Prompt constructed with constraints.
3. Request sent to Gemini API.
4. JSON response validated.
5. Parsed into internal format.

Security Considerations:

- API keys stored as environment variables.
- Keys managed via Hugging Face Secrets.
- No hardcoding of credentials.
- No storage of user data.

This ensures secure and ethical AI deployment.

11.6 OCR System Integration

Image-based blood reports require OCR processing.

System-level dependency:

- Tesseract OCR engine installed via packages.txt.

Python wrapper:

- pytesseract

Integration Flow:

Image → OCR → Extracted Text → Regex Parser → Structured Data

Handling OCR at deployment level ensures portability across environments.

11.7 PDF Report Generation

The system generates downloadable PDF reports using ReportLab.

Process:

1. Final synthesized report generated.

2. Text formatted into PDF layout.
3. File saved in temporary directory.
4. Returned to user via download component.

This feature improves professionalism and real-world usability.

11.8 Cloud Build Configuration

The deployment includes:

- requirements.txt for Python dependencies
- packages.txt for system-level dependencies (e.g., Tesseract)

During container build:

- Python packages installed
- OCR engine installed
- Gradio app launched

This ensures reproducibility and environment consistency.

11.9 System Integration Workflow

The complete deployed pipeline operates as follows:

1. User uploads file.
2. Backend detects file type.
3. Model 1 extracts and classifies parameters.
4. Model 2 computes dynamic risk scores via API.
5. Model 3 synthesizes final report.
6. Model 4 enables interactive Q&A.
7. PDF generated for download.
8. Results displayed on dashboard.

All modules operate within a single orchestrated flow.

11.10 Performance Considerations

- API response latency depends on Gemini inference time.
- OCR processing time varies with image resolution.
- Overall average processing time remains within interactive threshold.

11.11 Deployment Advantages

- Fully cloud-based
- No local installation required
- Easily shareable public URL
- Scalable to larger infrastructure
- Suitable for research demonstration

11.12 Limitations in Deployment

- Internet connectivity required
- API rate limits may apply
- Performance dependent on cloud execution environment

Despite these constraints, the deployed system demonstrates real-world viability.

12. Implementation Details

This section describes the practical implementation of the proposed Multi-Model AI Agent. The system is implemented in Python using a modular architecture, where each file corresponds to a specific functional component. The implementation ensures separation of concerns, maintainability, and extensibility.

12.1 Overall Project Structure

The project directory structure is organized as follows:

```
app.py
file_parser.py
model.py
gemini_agent.py
synthesis_agent.py
qa_agent.py
pdf_generator.py
requirements.txt
packages.txt
```

Each module is described below.

12.2 file_parser.py – Data Extraction Module

This module handles multi-format file ingestion and text extraction.

Functional Responsibilities:

- Detect file type
- Extract text from PDF
- Perform OCR on images
- Parse CSV files
- Read JSON data
- Normalize extracted content

Core Functions:

1. parse_uploaded_file(file)
    - Identifies file extension.

    - Routes to appropriate extractor.
    - Returns structured text data.
2. PDF Extraction:
    - Uses pdfplumber.
    - Extracts page-wise text.
3. Image Extraction:
    - Uses pytesseract.
    - Converts image to string.
4. CSV Parsing:
    - Uses pandas.read_csv.
5. JSON Handling:
    - Direct dictionary conversion.

This module standardizes all inputs into a unified format for Model 1.

12.3 model.py – Parameter Interpretation Module

This module implements Model 1 logic.

Core Functions:

1. analyze_report(report_data)
    - Iterates over structured parameter data.
    - Compares tested value against reference range.
    - Assigns Normal / High / Low status.
    - Returns structured dictionary.
2. generate_summary(model1_output)
    - Counts abnormal parameters.
    - Generates brief deterministic summary.

Design Characteristics:

- Deterministic logic
- No external API dependency

- Transparent classification rules
- Easily testable

12.4 gemini_agent.py – Risk Scoring Module

This module implements Model 2 logic.

Core Function:

gemini_risk_analysis(model1_analysis)

Steps:

1. Prepare structured JSON.
2. Construct detailed prompt.
3. Send request to Gemini API.
4. Receive structured JSON response.
5. Validate and parse output.

Error Handling:

- Catch invalid JSON.
- Handle API response errors.
- Return structured error message if failure occurs.

This module introduces probabilistic AI reasoning into the system.

12.5 synthesis_agent.py – Report Synthesis Module

Implements Model 3 logic.

Core Function:

generate_final_health_report(model1_output, model2_output)

Processing:

1. Combine abnormality data and risk scores.
2. Construct structured prompt.
3. Request narrative report from LLM.
4. Return formatted report text.

This module ensures interpretability and structured natural language generation.

12.6 qa_agent.py – Personalized Chat Module

Implements Model 4 logic.

Core Function:

ask_question_about_report(model1_data, model2_data, final_report, user_profile, question)

Processing Steps:

1. Merge:
    - Parameter data
    - Risk scoring
    - Synthesized report
    - User profile
2. Construct contextual prompt.
3. Send to Gemini API.
4. Generate personalized answer.

Key Design Features:

- Context injection
- Demographic-aware reasoning
- Controlled language constraints

12.7 pdf_generator.py – Report Export Module

Handles PDF generation using ReportLab.

Core Function:

generate_pdf(report_text)

Process:

1. Initialize PDF canvas.
2. Wrap long lines.

3. Add text page-by-page.
4. Save file in temporary directory.
5. Return file path.

This module enhances system professionalism.

12.8 app.py – Orchestration and UI Layer

This is the central control module.

Responsibilities:

- Define Gradio UI layout
- Manage state variables
- Connect UI events to backend functions
- Coordinate model execution pipeline

Workflow Inside app.py:

1. User uploads file.
2. Call file_parser.
3. Call analyze_report.
4. Call gemini_risk_analysis.
5. Call generate_final_health_report.
6. Display dashboard.
7. Enable chatbot.
8. Generate PDF.

Gradio Blocks is used for:

- Tab-based layout
- State management
- Event handling
- Interactive response updates

12.9 State Management

Gradio's gr.State() is used to store:

- Model 1 output
- Model 2 output
- Final report text

This enables Model 4 (chatbot) to access previous analysis without reprocessing.

12.10 Error Handling Strategy

The system includes:

- File validation checks
- JSON parsing validation
- API response validation
- Exception handling in each module
- User-friendly error messages

This improves robustness.

12.11 Dependency Management

Two files manage dependencies:

requirements.txt

For Python packages:

- gradio
- pandas
- pdfplumber
- pytesseract
- pillow
- reportlab
- google-generativeai

packages.txt

For system packages:

- tesseract-ocr

This ensures reproducible cloud deployment.

12.12 Implementation Strengths

- Modular design
- Clean separation of logic
- Scalable architecture
- Cloud-compatible
- API-driven intelligence
- Minimal hardcoding
- Structured data flow

13. Performance Evaluation and Experimental Results

This section evaluates the performance of the proposed Multi-Model AI Agent under various testing scenarios. The evaluation focuses on accuracy of extraction, reliability of risk estimation, response quality, system robustness, and deployment stability.

13.1 Evaluation Objectives

The system was evaluated based on the following criteria:

1. Accuracy of parameter extraction
2. Correctness of abnormality classification
3. Logical consistency of risk scoring
4. Coherence of synthesized reports
5. Quality of personalized chatbot responses
6. System response time
7. Deployment stability

13.2 Test Dataset Description

Since the system is designed for real-world usage rather than dataset-based training, evaluation was conducted using multiple sample blood reports including:

- Normal blood reports
- Reports with high glucose levels
- Reports with high cholesterol
- Reports with low hemoglobin
- Reports with multiple abnormal parameters
- Scanned image-based reports
- CSV-structured lab exports

These test cases simulated real-world variability.

13.3 Evaluation of Model 1 (Extraction & Classification)

Metric 1: Extraction Accuracy

The extraction module was tested on multiple file formats.

File Type	Extraction Success Rate
Text-based PDF	High
Scanned Image	Moderate to High (dependent on image clarity)
CSV	Very High
JSON	Very High

Observations:

- PDF parsing performed reliably for text-based reports.
- OCR accuracy depended on image resolution and clarity.
- CSV and JSON formats achieved near-perfect structured parsing.
- Regex patterns successfully identified most common parameter formats.

Abnormality Classification:

Classification logic was deterministic and showed 100% correctness when valid reference ranges were present.

13.4 Evaluation of Model 2 (Risk Scoring Engine)

Since Model 2 uses LLM-based probabilistic reasoning, evaluation focused on:

- Logical consistency
- Alignment with abnormal parameters
- Risk differentiation across test cases

Scenario 1: High Fasting Glucose

Observation:

- Diabetes risk increased significantly.
- Reasoning correctly referenced elevated glucose.

Scenario 2: High LDL and Triglycerides

Observation:

- Cardiovascular risk increased appropriately.

- Explanation referenced lipid abnormalities.

Scenario 3: Low Hemoglobin

Observation:

- Anemia risk elevated.
- Suggested hematology consultation.

Consistency Analysis:

Across multiple runs:

- Risk scores remained within reasonable medical plausibility.
- Qualitative risk levels aligned with abnormal severity.
- No diagnostic claims were generated.

13.5 Evaluation of Model 3 (Report Synthesis)

Evaluation criteria:

1. Clarity
2. Readability
3. Logical prioritization
4. Actionability

Findings:

- Reports included structured sections.
- Top abnormal findings were correctly prioritized.
- Recommendations were actionable (diet, exercise, follow-up tests).
- Language was supportive and non-alarming.
- Disclaimer consistently appended.

The synthesized reports were suitable for non-medical users.

13.6 Evaluation of Model 4 (Personalized Q&A)

Evaluation was performed using different demographic inputs.

Test Case A:

Age: 22
High glucose

Observation:

- Response emphasized lifestyle modification.
- Lower urgency tone.

Test Case B:

Age: 65
High glucose

Observation:

- Greater emphasis on monitoring.
- Suggested physician consultation more strongly.

Test Case C:

User asked symptom-based question

Observation:

- AI incorporated symptom context.
- Provided contextual explanation.

Conclusion:

The chatbot demonstrated context-aware personalization and safe medical communication.

13.7 System Performance Metrics

Response Time:

- Extraction: Fast for PDF and CSV
- OCR: Moderate (depends on image size)
- LLM inference: Dependent on API latency
- Overall average response time: Within interactive usability limits

Stability:

- No crashes during standard usage.
- Proper error handling for invalid files.
- API error messages displayed gracefully.

13.8 Robustness Testing

The system was tested under:

- Missing reference ranges
- Partial parameter extraction
- Invalid file uploads
- API temporary failures

Results:

- Graceful error handling implemented.
- Structured error messages returned.
- No fatal system crashes observed.

13.9 Limitations Observed During Testing

1. OCR performance reduced for low-resolution images.
2. Slight variation in LLM-generated text across runs.
3. Complex tabular report layouts may require advanced parsing.

These limitations are manageable and do not compromise system integrity.

13.10 Summary of Experimental Findings

The evaluation demonstrates that:

- The system reliably extracts and classifies parameters.
- LLM-based risk scoring is logically consistent.
- Report synthesis improves interpretability.
- Personalized Q&A enhances user engagement.
- Deployment environment remains stable.

Overall, the proposed Multi-Model AI Agent performs effectively as a Clinical Decision Support Tool for automated blood report interpretation.

14. Limitations and Ethical Considerations

Although the proposed Multi-Model AI Agent demonstrates strong performance in automated blood report interpretation, it is important to acknowledge its technical limitations and ethical implications. Healthcare-related AI systems must be developed and deployed responsibly to avoid misuse or misinterpretation.

This section outlines both technical limitations and ethical safeguards integrated into the system.

14.1 Technical Limitations

14.1.1 OCR Dependency on Image Quality

The accuracy of text extraction from scanned reports depends on:

- Image resolution
- Font clarity
- Lighting conditions
- Table formatting

Low-quality images may result in incomplete parameter extraction, which can affect downstream risk analysis.

14.1.2 Variability in Report Formats

Blood reports vary significantly across laboratories. Differences may include:

- Parameter naming conventions
- Layout structures
- Unit representations
- Multi-column formatting

While regex-based parsing handles common formats, highly irregular layouts may require advanced layout detection models.

14.1.3 Non-Deterministic LLM Output

Large Language Models generate probabilistic outputs. Therefore:

- Risk scores may vary slightly across runs.

- Wording of synthesized reports may differ.
- Responses are influenced by prompt structure.

Although structured prompting reduces variability, complete determinism is not guaranteed.

14.1.4 API Dependency

The system relies on external AI API services (Gemini API). Therefore:

- Internet connection is required.
- API rate limits may apply.
- Service downtime may affect functionality.

14.1.5 Limited Scope

Currently, the system focuses only on blood report analysis. It does not support:

- Radiology imaging
- ECG interpretation
- Genetic testing reports
- Complex multi-visit medical histories

Future enhancements may expand scope.

14.2 Ethical Considerations

Healthcare AI applications must prioritize safety, responsibility, and transparency. The following ethical safeguards are incorporated:

14.2.1 Non-Diagnostic Positioning

The system explicitly avoids:

- Declaring medical diagnoses
- Prescribing medications
- Replacing professional medical consultation

All outputs emphasize that the system provides risk estimation and informational support only.

14.2.2 Mandatory Disclaimer

Every generated report includes a disclaimer stating:

- The analysis is AI-based.
- It is not a medical diagnosis.
- Users should consult qualified healthcare professionals.

This ensures responsible usage.

14.2.3 Avoidance of Fear-Inducing Language

Prompt engineering ensures:

- Supportive tone
- Non-alarming phrasing
- Encouragement of medical consultation
- Avoidance of definitive claims

This minimizes unnecessary anxiety.

14.2.4 Data Privacy Considerations

The system:

- Does not store user-uploaded files permanently.
- Does not maintain long-term health records.
- Uses environment variables for API keys.
- Avoids hardcoding credentials.

This reduces privacy risk.

14.2.5 Transparency of AI Behavior

The system:

- Clearly separates deterministic logic (Model 1) from AI reasoning (Model 2–4).
- Displays structured risk scores and reasoning.

- Avoids hidden inference processes.

This improves interpretability and user trust.

14.3 Responsible AI Deployment in Healthcare

Healthcare AI systems must balance:

- Innovation
- Safety
- Transparency
- Ethical responsibility

The proposed system adheres to responsible AI principles by:

- Avoiding overreach
- Maintaining transparency
- Encouraging professional medical consultation
- Clearly defining system limitations

14.4 Summary of Limitations and Ethics

While the system demonstrates strong capability in automated health report interpretation, it is not intended to replace medical professionals. It serves as a decision-support and awareness tool designed to improve accessibility to health information.

By acknowledging technical limitations and embedding ethical safeguards, the system ensures responsible deployment of AI in a sensitive domain such as healthcare.

15. Future Scope and Research Extensions

While the proposed Multi-Model AI Agent demonstrates strong performance in automated blood report interpretation and personalized health reasoning, several opportunities exist for further enhancement and research expansion. This section outlines potential improvements in technical, architectural, and application domains.

15.1 Expansion to Multi-Modal Medical Reports

Currently, the system focuses on blood test reports. Future research can extend the architecture to support:

- Electrocardiogram (ECG) reports
- Radiology reports (X-ray, MRI, CT scans)
- Urine analysis reports
- Thyroid profiles
- Genetic testing summaries

This would require integration of:

- Image classification models
- Time-series signal analysis
- Domain-specific medical ontologies

The modular architecture of the current system makes such expansion feasible.

15.2 Integration of Deep Learning-Based OCR

The current OCR pipeline uses Tesseract, which may struggle with complex layouts. Future improvements could include:

- Transformer-based OCR models
- Layout-aware document parsing
- Vision-language models for table detection
- Deep learning document segmentation

This would enhance extraction accuracy for complex reports.

15.3 Personalized Risk Modeling Using Structured ML

Currently, risk scoring is LLM-based and probabilistic. Future research could:

- Train supervised ML models on structured medical datasets
- Combine LLM reasoning with predictive ML models
- Incorporate epidemiological data
- Introduce Bayesian risk modeling

A hybrid ML + LLM approach could improve quantitative precision.

15.4 Longitudinal Health Tracking

The current system analyzes single reports. Future extensions could include:

- Storing historical test data securely
- Tracking trends over time
- Generating time-series visualizations
- Predicting disease progression

This would transform the system into a longitudinal health monitoring platform.

15.5 Integration with Electronic Health Records (EHR)

Future work may include:

- Secure API integration with hospital systems
- Automated lab result fetching
- Patient history contextualization
- Real-time clinical decision support

This would enable institutional deployment.

15.6 Multilingual Support

To improve accessibility:

- Add support for regional languages
- Translate medical explanations into Hindi and other languages

- Provide culturally contextualized advice

This enhances inclusivity.

15.7 Explainable AI Enhancements

Although the system provides reasoning, future work could include:

- Confidence scores
- Visual explanation graphs
- Parameter impact weighting
- Explainability dashboards

This would strengthen interpretability and user trust.

15.8 Mobile and Edge Deployment

Future implementation could involve:

- Android/iOS mobile application
- Lightweight inference optimization
- Edge-based OCR processing
- Offline mode with limited functionality

This would improve real-world adoption.

15.9 Integration with Wearable Devices

Future research could integrate:

- Smartwatch health data
- Continuous glucose monitoring
- Heart rate variability
- Sleep patterns

Combining lab data with wearable data would provide holistic health analytics.

15.10 Advanced AI Safety Enhancements

To further strengthen responsible AI usage:

- Introduce medical validation layers
- Add rule-based safety checks before displaying outputs
- Integrate professional review pipelines

This would improve reliability in high-risk scenarios.

15.11 Research Contributions and Scalability Potential

The modular multi-model architecture provides a scalable framework for:

- AI-driven medical analytics
- Personalized health assistants
- Clinical decision support tools
- Preventive healthcare monitoring

The architecture can be generalized beyond blood reports to other structured medical domains.

15.12 Summary of Future Directions

The proposed system serves as a foundational AI-driven healthcare assistant. Future enhancements may:

- Improve extraction accuracy
- Expand diagnostic coverage
- Integrate structured ML models
- Enable longitudinal tracking
- Increase deployment scale

16. Conclusion

This research presented the design and implementation of a **Multi-Model AI Agent for Automated Health Diagnostics**, focused on intelligent interpretation of blood test reports. The system integrates structured data extraction, deterministic parameter classification, dynamic large language model-based risk assessment, natural language report synthesis, and personalized contextual interaction within a unified modular architecture.

The primary objective of the research was to bridge the gap between raw laboratory data and meaningful, accessible health insights for non-medical users. By combining deterministic logic (Model 1) with probabilistic AI reasoning (Models 2–4), the system demonstrates how hybrid AI architectures can enhance interpretability, scalability, and usability in healthcare applications.

The proposed solution successfully:

- Extracts medical parameters from multi-format reports (PDF, image, CSV, JSON).
- Identifies abnormal findings using deterministic threshold-based logic.
- Computes dynamic risk scores across multiple disease categories using LLM reasoning.
- Generates structured, human-readable health summaries.
- Provides personalized question-answering based on demographic and lifestyle inputs.
- Deploys as a fully functional cloud-based application.

The modular design ensures that each component operates independently while contributing to the overall system workflow. This separation of concerns improves maintainability, extensibility, and future scalability. The deployment on a cloud platform further demonstrates real-world feasibility.

While the system does not replace professional medical diagnosis, it functions effectively as a Clinical Decision Support Tool (CDSS) aimed at increasing health awareness, improving accessibility to medical insights, and facilitating informed discussions with healthcare providers.

From a research perspective, this work contributes to the integration of Large Language Models into healthcare analytics in a structured and responsible manner. It demonstrates that generative AI, when combined with deterministic preprocessing and ethical safeguards, can serve as a practical tool in medical data interpretation.

The research also highlights the importance of responsible AI deployment in healthcare, including transparency, non-diagnostic positioning, data privacy, and ethical safeguards.

In conclusion, the proposed Multi-Model AI Agent establishes a scalable, modular, and deployable framework for automated health diagnostics. The architecture provides a strong foundation for future extensions into broader medical domains, longitudinal monitoring, and advanced explainable AI systems.